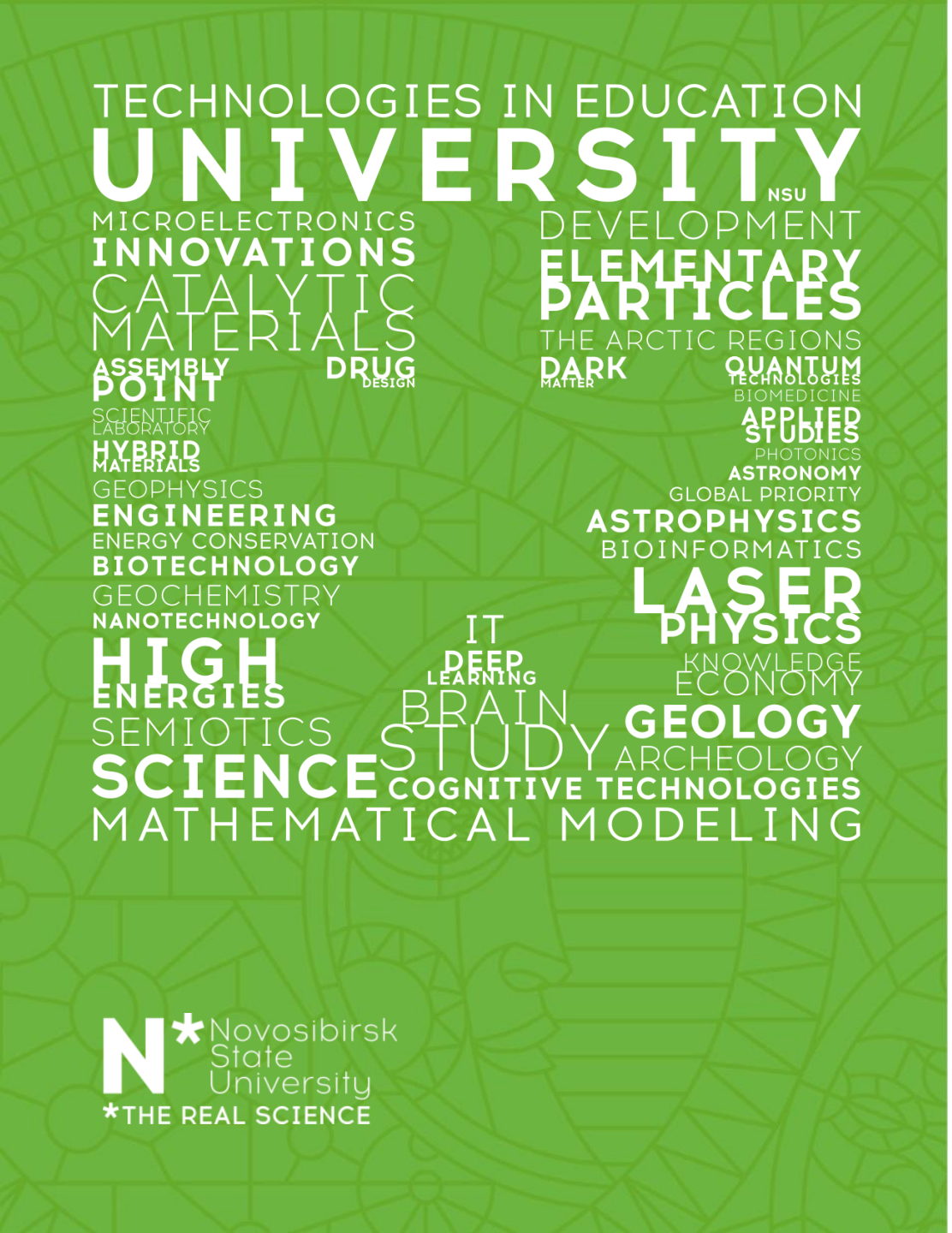
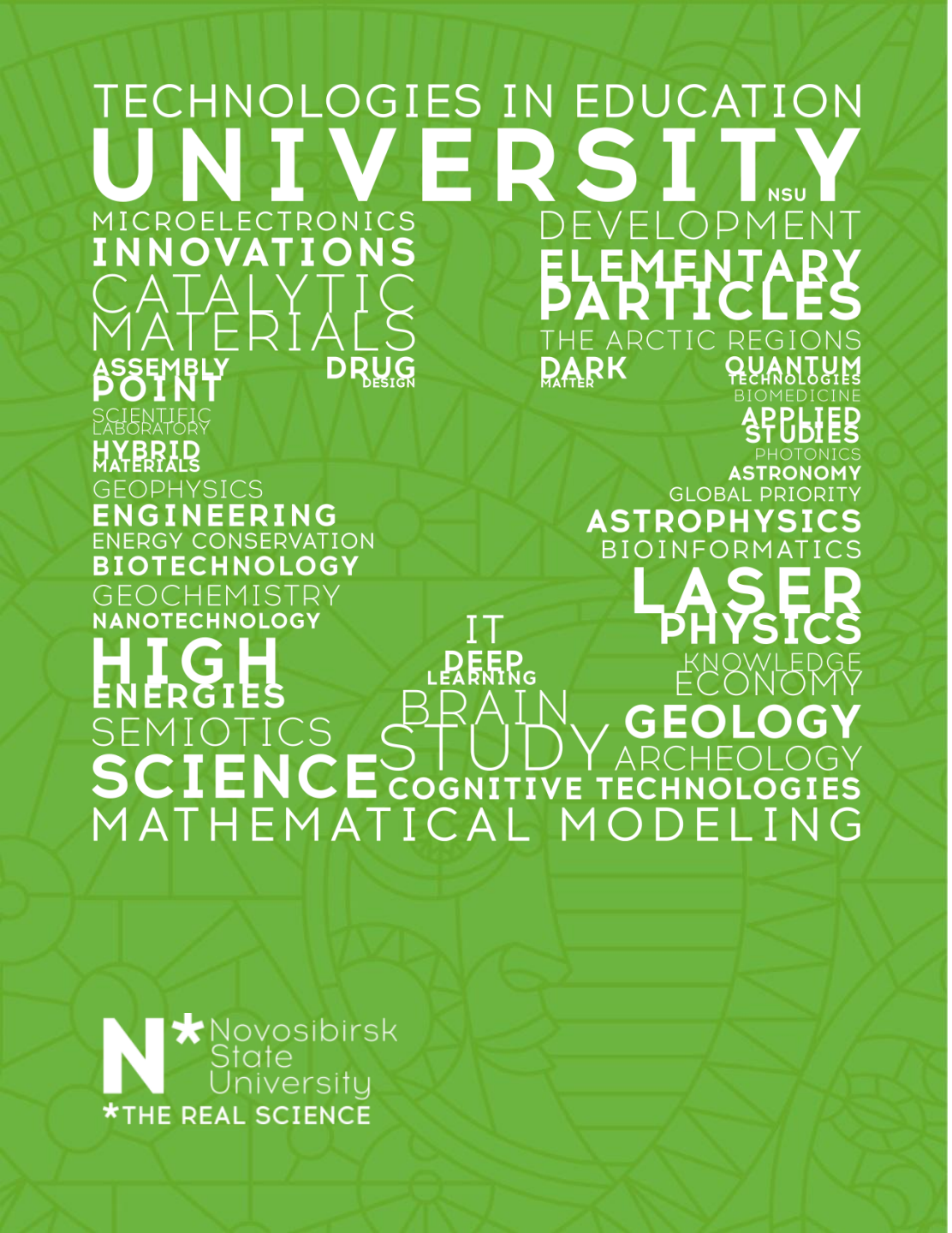


Машинное обучение
Семинар 10

Глушенко Андрей Валерьевич
Институт Интеллектуальной Робототехники



Машинное обучение ли?
Семинар 10

Глушенко Андрей Валерьевич
Институт Интеллектуальной Робототехники

Практические аспекты

Недообучение и переобучение

Разделение на обучающую, валидационную и тестовую выборки

Кросс-валидация

Метрики

Признаки плохого проекта

Жесткость: Систему сложно изменить, так как любое небольшое изменение запускает «эффект снежного кома», затрагивающий другие компоненты системы.

Хрупкость: При внесении изменений система ломается в областях, не связанных напрямую с модифицированным компонентом.

Неподвижность: Сложно разделить систему на компоненты, которые можно повторно использовать в других системах.

Вязкость: Сделать что-то правильно гораздо сложнее, чем выполнить неправильные действия.

Неоправданная сложность: Проект включает инфраструктуру, которая не приносит немедленной пользы.

Неопределенность: Проект сложно читать и понимать. Содержание проекта нечетко выражено.

Принципы проектирования ПО

Принципы проектирования подсказывают нам, как распределять функции и структуры данных между классами и как эти классы должны быть взаимосвязаны.

Использование слова «класс» не означает, что эти принципы применимы только к объектно-ориентированному программному обеспечению. Класс — это просто связанная группа функций и данных. В каждой программной системе есть такие группы, независимо от того, называются они классами или нет.

Принципы проектирования ПО

- ✓ SOLID
- ✓ Принципы комбинируемости компонентов: REP, CCP, CRP, ADP, SDP, SAP
- ✓ GOF
- ✓ GRASP
- ✓ YAGNI
- ✓ KISS
- ✓ DRY
- ✓ DDD

Важные принципы

- **YAGNI:** You Ain't Gonna Need It (Вам это не понадобится). Избегайте реализации функциональности, которая не является немедленно необходимой.
- **KISS:** Keep It Simple, Stupid (Держись простоты, глупец). Решения должны быть максимально простыми и понятными, минимизируя сложность.
- **DRY:** Don't Repeat Yourself (Не повторяйся). Исключите дублирование, выделив общий код в многократно используемые компоненты.
- **SOLID:** Набор принципов, способствующих хорошему проектированию программного обеспечения, включая принцип единственной ответственности, принцип открытости/закрытости, принцип подстановки Лискова, разделение интерфейсов и инверсию зависимостей.

KISS

- Принцип KISS (Keep It Simple, Stupid — Делай проще, глупый) гласит, что решения должны быть максимально простыми и понятными. Цель принципа — избегать создания чрезмерно сложных решений и упростить процесс разработки и сопровождения программного обеспечения.
- Принцип KISS подразумевает, что решение должно быть простым для понимания и сопровождения, а также минимальным с точки зрения количества компонентов и сложности взаимодействия между ними. Принцип KISS также означает, что разработчики должны выбирать самые простые решения, если они могут быть реализованы в рамках поставленной задачи.
- Принцип KISS важен для обеспечения читаемости кода, расширяемости, переносимости и тестируемости. Разработчики, следующие принципу KISS, должны сосредоточиться на минимизации сложности системы и упрощении интерфейсов, тем самым улучшая качество кода и снижая затраты на сопровождение.

KISS – пример сортировки пузырьком (Bubble Sort)

```
#KISS - Bubble Sort
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

YAGNI

- Принцип YAGNI (You Ain't Gonna Need It — Вам это не понадобится) гласит, что не следует реализовывать функциональность, которая в данный момент не требуется для решения конкретной проблемы.
- Этот принцип предостерегает от создания сложных и ненужных функций и компонентов, которые не только отвлекают от основной задачи, но и могут создать проблемы в будущем. Вместо этого следует сосредоточиться на реализации функций, которые действительно необходимы, а новая функциональность должна добавляться только тогда, когда это необходимо для удовлетворения новых требований.
- Принцип YAGNI направлен на экономию времени и ресурсов за счет избегания реализации ненужных функций и помогает сосредоточиться на основной задаче. Он также помогает избежать проблем, связанных с поддержкой неиспользуемых функций в будущем, что может привести к увеличению сложности кода и снижению его качества.

YAGNI – пример на Python

```python
#YAGNI - PROBLEM
def add_numbers(a, b):
    return a + b

def subtract_numbers(a, b):
    return a - b

def multiply_numbers(a, b):
    return a * b

def divide_numbers(a, b):
    return a / b

def calculate(a, b, operation):
    if operation == "add":
        return add_numbers(a, b)
    elif operation == "subtract":
        return subtract_numbers(a, b)
    elif operation == "multiply":
        return multiply_numbers(a, b)
    elif operation == "divide":
        return divide_numbers(a, b)
    else:
        print("Invalid operation")
        return None
```

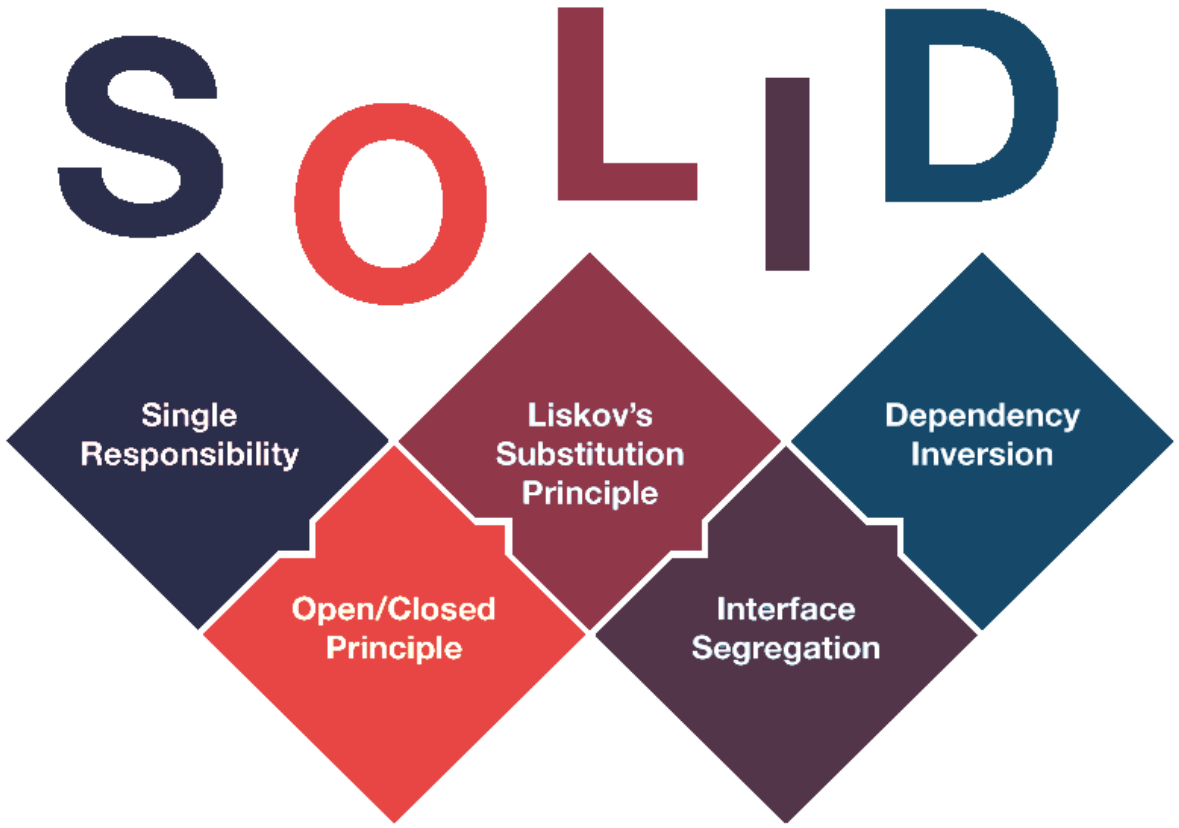
```python
#FIX
def add_numbers(a, b):
    return a + b

def subtract_numbers(a, b):
    return a - b

def calculate(a, b, operation):
    if operation == "add":
        return add_numbers(a, b)
    elif operation == "subtract":
        return subtract_numbers(a, b)
    else:
        print("Invalid operation")
        return None
```

DRY

- Принцип DRY (Don't Repeat Yourself — не повторяйся) гласит, что каждый фрагмент кода должен иметь единственное, однозначное представление в системе. Это означает, что при написании кода следует избегать его дублирования в других местах, где он может быть использован.
- Дублирование кода может привести к ряду проблем, включая увеличение сложности сопровождения, потенциальную возможность ошибок, усиление взаимосвязи между различными частями кода, трудности при внесении изменений в систему и многое другое. Принцип DRY помогает избежать этих проблем, создавая код, который легко поддерживать и модифицировать, а также улучшая его качество и читаемость.
- Принцип DRY может быть реализован различными способами, такими как использование функций и классов, которые могут вызываться из разных частей системы, избегание дублирования логики, выделение общего кода в отдельные модули или библиотеки и так далее.
- Принцип DRY является одним из фундаментальных принципов в разработке программного обеспечения и считается хорошей практикой в разработке ПО.

DRY – пример на Python

```python
#DRY - PROBLEM
students = [
    {"name": "Alice", "age": 20, "major": "Computer Science", "gpa": 3.5},
    {"name": "Bob", "age": 19, "major": "Mathematics", "gpa": 3.2},
    {"name": "Charlie", "age": 21, "major": "English", "gpa": 3.8},
    {"name": "Dave", "age": 22, "major": "Physics", "gpa": 3.6},
    {"name": "Eve", "age": 20, "major": "History", "gpa": 3.0}
]

print("Name: " + students[0]["name"] + ", Age: " + str(students[0]["age"]) + ", Major: " + students[0]["major"] + ", GPA: " +
str(students[0]["gpa"]))
print("Name: " + students[1]["name"] + ", Age: " + str(students[1]["age"]) + ", Major: " + students[1]["major"] + ", GPA: " +
str(students[1]["gpa"]))
print("Name: " + students[2]["name"] + ", Age: " + str(students[2]["age"]) + ", Major: " + students[2]["major"] + ", GPA: " +
str(students[2]["gpa"]))
print("Name: " + students[3]["name"] + ", Age: " + str(students[3]["age"]) + ", Major: " + students[3]["major"] + ", GPA: " +
str(students[3]["gpa"]))
print("Name: " + students[4]["name"] + ", Age: " + str(students[4]["age"]) + ", Major: " + students[4]["major"] + ", GPA: " +
str(students[4]["gpa"]))


#fix
for student in students:
    print("Name: {}, Age: {}, Major: {}, GPA: {}".format(student["name"], student["age"], student["major"], student["gpa"]))
```


SOLID
Single Responsibility
Liskov's Substitution Principle
Dependency Inversion
Open/Closed Principle
Interface Segregation

SOLID

При совместном применении эти принципы призваны повысить вероятность того, что программист создаст систему, которая будет легко поддерживаться и расширяться в течение длительного периода времени.

Принципы SOLID служат ориентирами, которые можно применять в процессе разработки программного обеспечения для устранения ненужного кода, побуждая программиста рефакторить исходный код до тех пор, пока он не станет хорошо написанным и расширяемым.

Цель этих принципов — создание промежуточных программных структур, которые:

- Устойчивы к изменениям
- Легко понятны
- Могут использоваться в качестве основы компонентов во многих программных системах.

SOLID

- **Принцип единственной ответственности (SRP):** Каждый программный модуль должен иметь одну-единственную причину для изменения.
- **Принцип открытости-закрытости (OCP):** Программные системы должны быть спроектированы таким образом, что позволяет изменять их поведение путем добавления нового кода, а не путем изменения существующего кода.
- **Принцип подстановки Лискова (LSP):** Программные системы должны строиться из взаимозаменяемых частей, которые соответствуют контракту, позволяющему заменять одну часть другой.
- **Принцип разделения интерфейсов (ISP):** Избегайте зависимости от того, что вы не используете.
- **Принцип инверсии зависимостей (DIP):** Политика высокого уровня не должна зависеть от деталей реализации низкого уровня. Вместо этого детали должны зависеть от политик.

ООП (Объектно-ориентированное программирование)

Объектно-ориентированная декомпозиция

Программная система состоит из объектов, обменивающихся данными.
Каждый объект обладает:

- Поведением
- Состоянием
- Идентичностью (индивидуальностью)

Похожие объекты группируются в классы.

Класс 1

Класс 2

Ключевые концепты ООП

Класс — абстракция данных и поведения определенного «типа» объектов

Объект (экземпляр) — экземпляр класса в реальном времени в работающей программе.

Свойство — характеристика объекта, его параметр.

Метод — программа действий над объектом или его свойствами.

Событие — изменение состояния объекта.

Ключевые концепты ООП

Абстракция:

- Рассмотрение только существенных характеристик объекта для решения рассматриваемой проблемы.

Инкапсуляция:

- Скрытие деталей реализации, отделение договорных обязательств от их реализации.

Иерархия (наследование, обобщение):

- Упорядочивание абстракций, их размещение на разных уровнях.

Модульность (полиморфизм):

- Разделение системы на внутренне связанные модули, которые слабо связаны между собой.

Абстракция

- Позволяет нам создавать большие системы и поддерживать над ними контроль.
- Абстракция не является исключительно атрибутом ООП.
- Мы можем выносить суждения о материалах, не углубляясь в детали их молекулярной структуры.
- Абстракция — это способ извлечь набор значимых характеристик объекта, исключая незначительные.

Абстракция. Пример на Python

```python
from abc import ABC, abstractmethod

class Vehicle(ABC):
    def __init__(self, name):
        self.name = name

    @abstractmethod
    def move(self):
        pass

class Car(Vehicle):
    def move(self):
        print(f"The car {self.name} moves on the road.")

class Boat(Vehicle):
    def move(self):
        print(f"The boat {self.name} sails on the water.")
```

```python
# Usage example
my_car = Car("Ford Focus")
my_car.move()  # Output: The car Ford Focus moves on the road.

my_boat = Boat("Sea Ray")
my_boat.move()  # Output: The boat Sea Ray sails on the water.
```

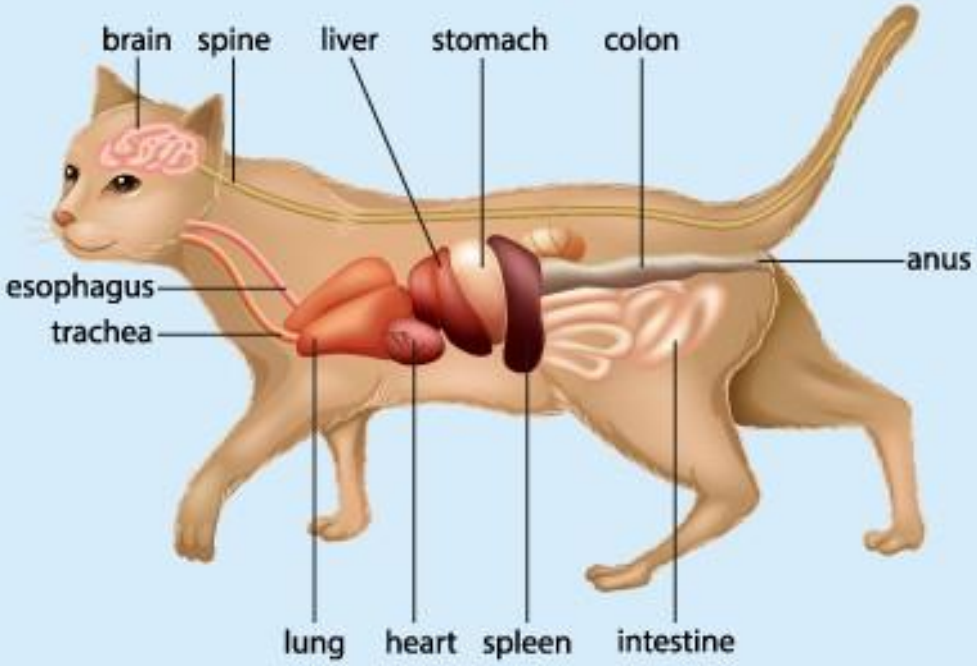
Инкапсуляция

- Скрытие реализации и внутренних процессов.
- Инкапсуляция — это свойство системы, позволяющее объединять данные и методы, работающие с ними, в классе и скрывать детали реализации от пользователя/другого разработчика/самого себя.
- Инкапсуляция тесно связана с концепцией интерфейса класса.

Инкапсуляция

Понятно, что делать с объектом
(кормить, что же еще)

Не понятно, как именно взаимодействовать с объектом
(слишком много деталей)



Инкапсуляция. Пример на Python

```python
class BankAccount:
    def __init__(self, account_number, name, balance):
        self.__account_number = account_number  # Private attribute
        self.__name = name                      # Private attribute
        self.__balance = balance                # Private attribute

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            self.__show_balance()

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            self.__show_balance()
        else:
            print("Insufficient funds in the account.")

    def __show_balance(self):                   # Private method
        print(f"Current balance: {self.__balance}")

    # Protected method - uses single underscore
    def _get_balance(self):
        return self.__balance

    # Method to get account information
    def get_account_info(self):
        return f"Account number: {self.__account_number}, Owner's name:
{self.__name}"
```

```python
# Example usage:
account = BankAccount('1234', 'John Doe', 1000)
account.deposit(500)
account.withdraw(200)
```

Иерархия / Наследование

- Наследование — это системное свойство, позволяющее описывать новый класс на основе существующего, частично или полностью заимствуя его функциональность.
- Класс, от которого происходит наследование, называется базовым или родительским классом. Новый класс является дочерним, потомственным или производным классом.
- На практике механизм наследования используется для уточнения (а не расширения!) функциональности существующих сущностей без копирования кода.

Инкапсуляция. Пример на Python

```python
class Animal:

    def __init__(self, species):
    self.species = species

    def make_sound(self):
    pass


class Dog(Animal):

    def __init__(self, name, species):
    super().__init__(species)
    self.name = name

    def make_sound(self):
    return "Woof!"


class Cat(Animal):

    def __init__(self, name, species):
    super().__init__(species)
    self.name = name


    def make_sound(self):
    return "Meow!"
```

```python
# Create instances of Dog and Cat
buddy = Dog(name="Buddy", species="Canine")
whiskers = Cat(name="Whiskers", species="Feline")

# Use inherited method make_sound
print(f"{buddy.name} the {buddy.species} says
{buddy.make_sound()}")
print(f"{whiskers.name} the {whiskers.species} says
{whiskers.make_sound()}")

# Output:
# Buddy the Canine says Woof!
# Whiskers the Feline says Meow!
```

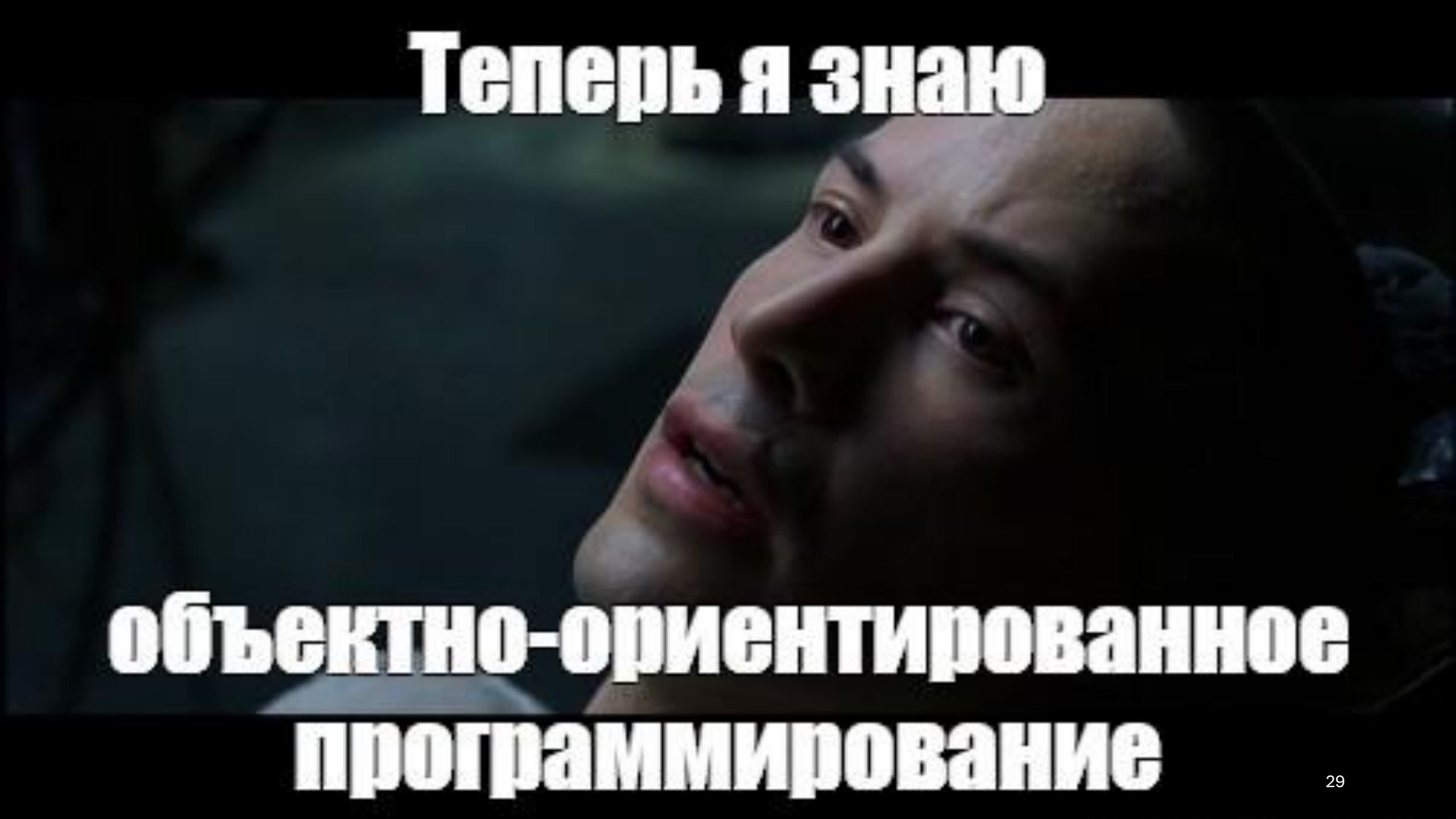
Полиморфизм

- **Полиморфизм** — это системное свойство использования объектов с общим интерфейсом без знания типа объекта и его внутренней структуры.
- Интерфейс описывает, что должен содержать каждый класс, реализующий его.
- Программист может забыть о существовании этих классов и работать только с их общим интерфейсом, не обращая внимания на тип объекта. Это упрощает понимание кода.

Полиморфизм. Пример на Python

```python
class Animal:
    def talk(self):
        pass

class Dog(Animal):
    def talk(self):
        return "Woof!"

class Cat(Animal):
    def talk(self):
        return "Meow!"

class Cow(Animal):
    def talk(self):
        return "Moo!"
```

```python
# Create a list of animals
animals = [Dog(), Cat(), Cow()]

# Make the animals talk
for animal in animals:
    print(animal.talk())
```


Теперь я знаю
объектно-ориентированное
программирование

Характеристики архитектуры:

Архитектура состоит из:

- Набора структурных элементов системы
- Спецификации поведения этих элементов и взаимосвязей между ними
- Структуры и поведения все более крупных подсистем, построенных на основе этих элементов
- Общие соображения по построению системы, такие как методы интеграции

Архитектор программного обеспечения

Ответственность:

- Технологический аспект программного проекта.
- Качество реализации программного продукта.
- Определение способа построения системы.

ИТ-архитектор и архитектор ИБ планируют
инфраструктуру в 2026 году

- **Конечный пользователь** заинтересован в интуитивно понятном и корректном поведении, производительности, надежности, простоте использования, доступности и безопасности;
- **Системный администратор** заинтересован в интуитивно понятном поведении, инструментах управления и мониторинга;
- **Специалист по маркетингу** заинтересован в конкурентных преимуществах, сроках выхода на рынок, позиционировании среди других продуктов и стоимости;
- **Клиент** заинтересован в цене, стабильности и возможности планирования;
- **Разработчик** заинтересован в четких требованиях и простом и последовательном принципе проектирования;
- **Менеджер проекта** заинтересован в предсказуемости процесса проектирования, планировании, продуктивном использовании ресурсов и бюджете;
- **Специалист службы поддержки** заинтересован в понятном принципе проекта, а также в простоте внесения изменений.